
AWS Aurora Serverless v2 – Full Lab Guide (Console-Based)

Goal

Deploy an **Aurora Serverless v2** database, connect using an SQL client, and **observe auto-scaling** in action.

Prerequisites

Requirement	Notes
AWS Account	Required
VPC with private subnets	Default VPC is OK (Aurora must be in a VPC)
Bastion Host or EC2 for MySQL client	For testing connection
IAM user/role with RDS full access	For lab creation

Step 1: Create Aurora Serverless v2 DB Cluster

1. **Go to:** AWS Console → **Amazon RDS**
2. Click **Create database**

3. Choose Engine options:

- Engine type: ☒ **Amazon Aurora**
- Edition: ☒ **Amazon Aurora MySQL-Compatible**
- Version: Choose latest compatible (e.g., 8.0+)
- Template: **Dev/Test**

4. DB Cluster Settings:

- DB cluster identifier: **aurora-serverless-v2-demo**
- Master username: **admin**
- Password: **YourStrongPassword123!**

5. DB Instance Configuration:

- Capacity type: ☒ **Serverless**
- Enable Aurora Serverless v2 ☒
- Min ACUs: **0.5**
- Max ACUs: **2** (adjust later to test scaling)

6. Connectivity:

- Virtual Private Cloud (VPC): Select existing/default
- Subnet group: Choose existing (must have 2+ AZs)
- Public access: ☒ **Yes** (or No if using a Bastion)

- VPC security group: Allow port **3306 (MySQL)**

7. **Additional Configuration:**

- Backup, monitoring: Leave defaults
- Enable Data API: Optional (for querying via AWS SDK)

8. Click **Create database**



Wait for Status: Available (~3–5 minutes)



Step 2: Connect to the DB

Option A: Use EC2 as a MySQL Client

1. Launch a small EC2 instance (Amazon Linux 2)

Install MySQL client:

```
sudo yum install -y mysql
```

- 2.

Connect using endpoint from RDS Console:

```
mysql -h <endpoint> -u admin -p
```

3. You'll be prompted for the password
-

Step 3: Load Test & Trigger Auto-Scaling

Now let's **simulate database load** to see Aurora scale automatically.

Option A: Use **sysbench** (on EC2)

Install **sysbench**:

```
sudo amazon-linux-extras enable epel  
sudo yum install -y sysbench
```

1.

Prepare the test DB:

```
sysbench oltp_read_write \  
--db-driver=mysql \  
--mysql-user=admin \  
--mysql-password=YourStrongPassword123! \  
--mysql-host=<aurora-endpoint> \  
--mysql-db=sbtest \  
--tables=10 \  
--table-size=10000 prepare
```

2.

Run load:

```
sysbench oltp_read_write \  
--db-driver=mysql \  
--mysql-user=admin \  
--mysql-password=YourStrongPassword123! \  
--mysql-host=<aurora-endpoint> \  
--mysql-db=sbtest \  
--tables=10 \  
--table-size=10000 \  
--threads=50 \  
--time=300 run
```

3.  Run for 5 minutes to observe scaling.

Option B: Use Apache Bench (ab)

`ab -n 1000 -c 100 http://<api-endpoint-if-using-data-api>`

Step 4: Monitor Auto-Scaling

Go to:

RDS → Databases → aurora-serverless-v2-demo → Monitoring

Look for:

- **ACU Usage** over time
- **Connections**
- **CPU Utilization**

 Aurora Serverless v2 will **scale ACUs based on load** (up to Max ACU you set earlier).

Optional Tuning

To simulate scaling more aggressively:

1. Go to DB Cluster → Modify
2. Increase **Max ACU** to 8 or 16

3. Apply changes **immediately**
 4. Re-run `sysbench` or `ab`
-

Step 5: Clean Up (Post-Lab)

To avoid charges:

- Stop or delete the EC2 instance
 - Delete the Aurora Serverless cluster:
 - Go to RDS → Databases → Select cluster → Actions → **Delete**
-

Summary Table

Component	Purpose
Aurora Serverless v2	Auto-scaling RDS DB
ACU	Aurora Capacity Unit (1 ACU = ~2 GB RAM & CPU)
MySQL Client	To connect and test
Sysbench / ab	To simulate load
Monitoring tab	Observe auto-scaling metrics
